

GRAVEATLAS

Phase 1 — Project Architecture & Foundation

Master implementation prompt compiled from the original Phase 1 specification.

PROJECT ARCHITECTURE

Core flow: Android App → Cloudflare Worker/API → GitHub App → Public GitHub Data Repository, with pending contributions passing through moderation before publication.

PHASE 1 OBJECTIVES

Establish the complete technical foundation for GraveAtlas without prematurely adding later-phase features. The implementation must be modular, secure, testable, documented, and ready for later phases.

COPY-PASTE MASTER PROMPT

GRAVEATLAS — PHASE 1
PROJECT ARCHITECTURE & FOUNDATION

Build the foundation of GraveAtlas.

CORE ARCHITECTURE

1. Android application is the primary user-facing client.
2. Cloudflare Worker/API is the backend gateway.
3. GitHub App provides controlled repository access.
4. Public cemetery/grave data is maintained in a public GitHub data repository.
5. User submissions do NOT immediately modify the public dataset.
6. Contributions enter a pending/moderation workflow.
7. Approved information can then be published to the public data repository.

ANDROID FOUNDATION

- Create a clean modular Android project.
- Establish package/application identity for GraveAtlas.
- Configure development and production environments separately.
- Create the basic navigation structure.
- Create a main/home screen foundation.
- Create search and cemetery/record viewing foundations.
- Create contribution entry points without implementing unrestricted publishing.
- Implement loading, empty, offline and error states.
- Keep all secrets out of the Android application.

CLOUDFLARE WORKER / API FOUNDATION

- Create the backend Worker/API structure.
- Define versioned API routes.
- Validate all incoming requests.
- Implement consistent JSON responses.
- Implement safe error handling.
- Separate public, authenticated, moderation and administrative operations.
- Never expose private credentials or internal secrets.
- Establish request identifiers/logging where appropriate.

GITHUB APP FOUNDATION

- Use the existing GitHub App.
- Configure the App ID, private key and installation ID through secure environment/secret storage.
- Never hard-code the private key.
- Never expose GitHub credentials to Android clients.
- Use the GitHub App only from the trusted backend.
- Verify repository and installation permissions.
- Establish controlled read/write operations required by the architecture.

DATA MODEL FOUNDATION

Define schemas for the core entities, including as applicable:

- cemetery
- grave/record
- person
- location/coordinates
- source/provenance
- contribution
- correction
- moderation status

- audit event

Every public record must have a clear provenance/source model.

Do not invent historical information.

PENDING / MODERATION WORKFLOW

Implement the foundation for:

1. User creates a contribution.
2. Backend validates it.
3. Contribution is stored as pending.
4. Moderation can review it.
5. Approved information is prepared for publication.
6. Publication is performed through the trusted backend/GitHub App.
7. Audit information records the operation.

Users must never receive unrestricted GitHub repository write access.

PUBLIC DATA REPOSITORY

Establish the repository structure for public data.

Use machine-readable data formats and a predictable directory structure.

Keep:

- data
- schemas
- documentation
- validation rules

separate where practical.

Do not place:

- secrets
- private user data
- admin credentials
- private moderation notes

inside the public repository.

API ROUTES

Define the initial route structure for:

- health
- public cemetery search
- public cemetery/record retrieval
- contribution creation
- contribution status
- moderation operations
- publication operations

Only expose routes that are actually implemented.

SECURITY FOUNDATION

Implement:

- authentication boundaries
- authorization checks
- input validation
- rate-limit hooks/structure
- secure secret handling
- audit logging
- safe error responses
- CORS policy appropriate to the actual clients

Use least privilege.

Never expose:

- GITHUB_PRIVATE_KEY
- ADMIN_TOKEN
- Cloudflare credentials
- API keys
- authentication tokens

TESTING FOUNDATION

Create tests for:

- API validation
- authentication/authorization boundaries
- core data schemas
- contribution workflow
- moderation boundaries
- GitHub App integration boundaries
- public data retrieval

- error handling

Tests must be deterministic where possible.

DOCUMENTATION

Create/update:

- README.md
- docs/ARCHITECTURE.md
- docs/API.md
- docs/DATA-SCHEMA.md
- docs/SECURITY.md
- docs/CONTRIBUTION-WORKFLOW.md
- docs/GITHUB-APP.md
- docs/DEVELOPMENT.md

Document actual implementation only.

PHASE 1 ACCEPTANCE GATE

Verify:

- [] Android project foundation
- [] Cloudflare Worker/API foundation
- [] GitHub App integration foundation
- [] Public data repository structure
- [] Core schemas
- [] API routes
- [] Contribution workflow
- [] Moderation boundary
- [] Publication boundary
- [] Authentication/authorization boundaries
- [] Secret protection
- [] Validation
- [] Error handling
- [] Audit foundation
- [] Tests
- [] Documentation

FINAL REPORT

Return:

PHASE 1 STATUS: READY / NOT READY

ANDROID: PASS / FAIL

CLOUDFLARE API: PASS / FAIL

GITHUB APP: PASS / FAIL

DATA MODEL: PASS / FAIL

MODERATION: PASS / FAIL

PUBLICATION: PASS / FAIL

SECURITY: PASS / FAIL

TESTS: PASS / FAIL

DOCUMENTATION: PASS / FAIL

List:

- completed items
- blockers
- non-blocking issues
- actual test results

Do not invent successful results.

STOP after Phase 1.

Do not start Phase 2 without explicit instruction.

Project name: GraveAtlas

Original Phase 1 title: Kubur SG — Phase 1: Project Architecture & Foundation